Python Activity-Based Learning

1.

Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

Sol:

Problem statement

Calculate an electricity bill using progressive slabs: first 100 units at Rs. 1.50/unit, next 100 at Rs. 2.50/unit, and units above 200 at Rs. 4.00/unit. Add a fixed charge of Rs. 50.

Objectives

- Practice input, conditional statements, arithmetic, and progressive slab calculations.
- Calculate energy charge based on consumption slabs.
- Add fixed charges and display the final electricity bill.

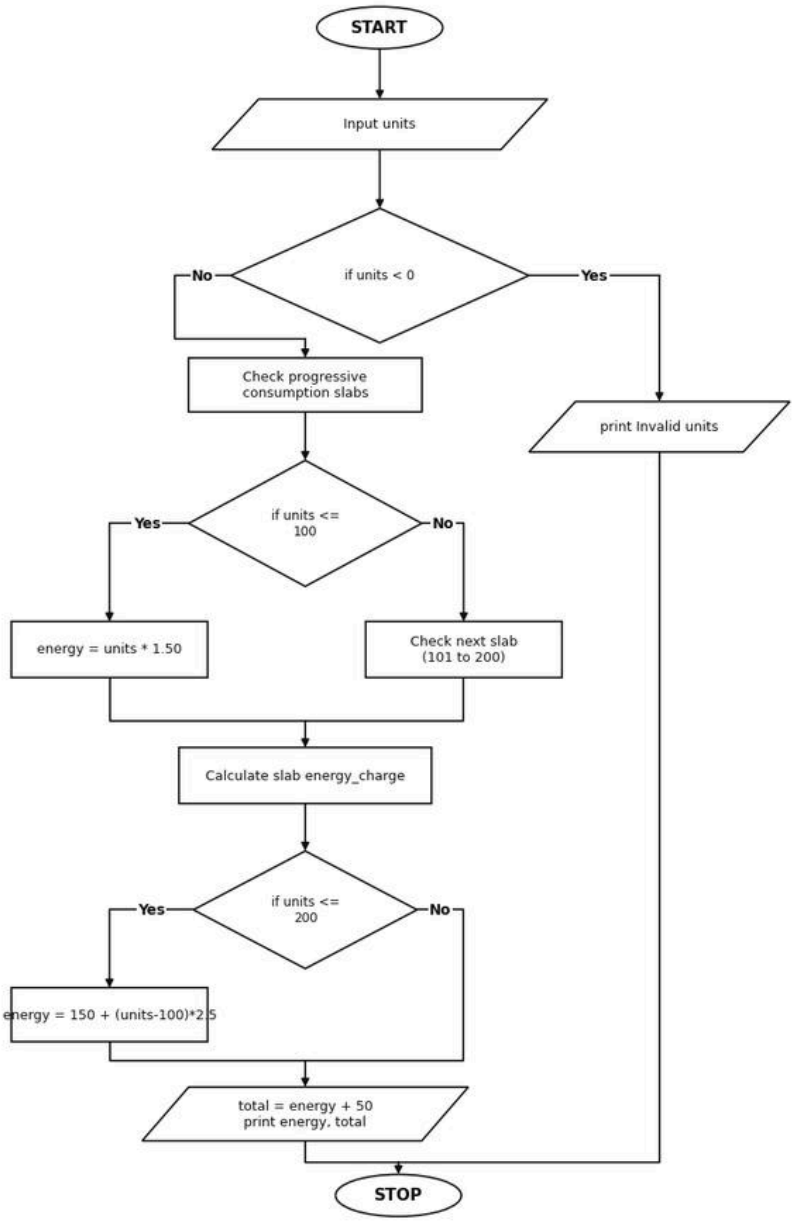
Assumptions

- Units consumed cannot be negative.
- Slab rates apply only to the units within each slab.
- The fixed charge of Rs. 50 is added once.

Algorithm

1. Start.
2. Read units consumed.
3. If units are negative, display invalid units.
4. If units are at most 100, charge all units at Rs. 1.50.
5. If units are 101 to 200, charge the first 100 at Rs. 1.50 and the remainder at Rs. 2.50.
6. If units exceed 200, charge the first 100 at Rs. 1.50, the next 100 at Rs. 2.50, and the rest at Rs. 4.00.
7. Add Rs. 50 fixed charge, display the bill, and stop.

Flowchart



Pseudocode

```
START
INPUT units
IF units < 0
   PRINT "Invalid units"
ELSE
   IF units <= 100
      energy_charge = units * 1.50
   ELSE IF units <= 200
```

```
        energy_charge = 100 * 1.50 + (units - 100) * 2.50
    ELSE
        energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
    total_bill = energy_charge + 50
    PRINT energy_charge, total_bill
STOP
```

Source Code

```
units = float(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 1.50
    elif units <= 200:
        energy_charge = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

    total_bill = energy_charge + 50
    print("Energy charge: Rs.", round(energy_charge, 2))
    print("Total bill: Rs.", round(total_bill, 2))
```

Test Cases

1. Enter units consumed: 50
   Output:
   Energy charge: Rs. 75.0
   Total bill: Rs. 125.0
2. Enter units consumed: 150
   Output:
   Energy charge: Rs. 275.0
   Total bill: Rs. 325.0

Output

Case_1_problem_2.py x

```python
# Case 1 - Problem 2: Progressive Electricity Billing

units = float(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 1.50
    elif units <= 200:
        energy_charge = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

    total_bill = energy_charge + 50
    print("Energy charge: Rs.", round(energy_charge, 2))
    print("Total bill: Rs.", round(total_bill, 2))
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_1_problem_2.py
Enter units consumed: 150
Energy charge: Rs. 275.0
Total bill: Rs. 325.0
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program calculates the electricity bill by applying each rate only to the units in its slab.

2.

Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

Sol:

Problem statement

Calculate a patient's hospital bill from room charges, consultation fee, and medicine cost. Classify the bill as Low (below Rs. 5,000), Moderate (Rs. 5,000 to Rs. 15,000), or High (above Rs. 15,000).

Objectives

- Use arithmetic and if-elif-else conditions to calculate and classify a hospital bill.
- Validate that entered hospital charges are non-negative.
- Display total hospital bill and patient category.

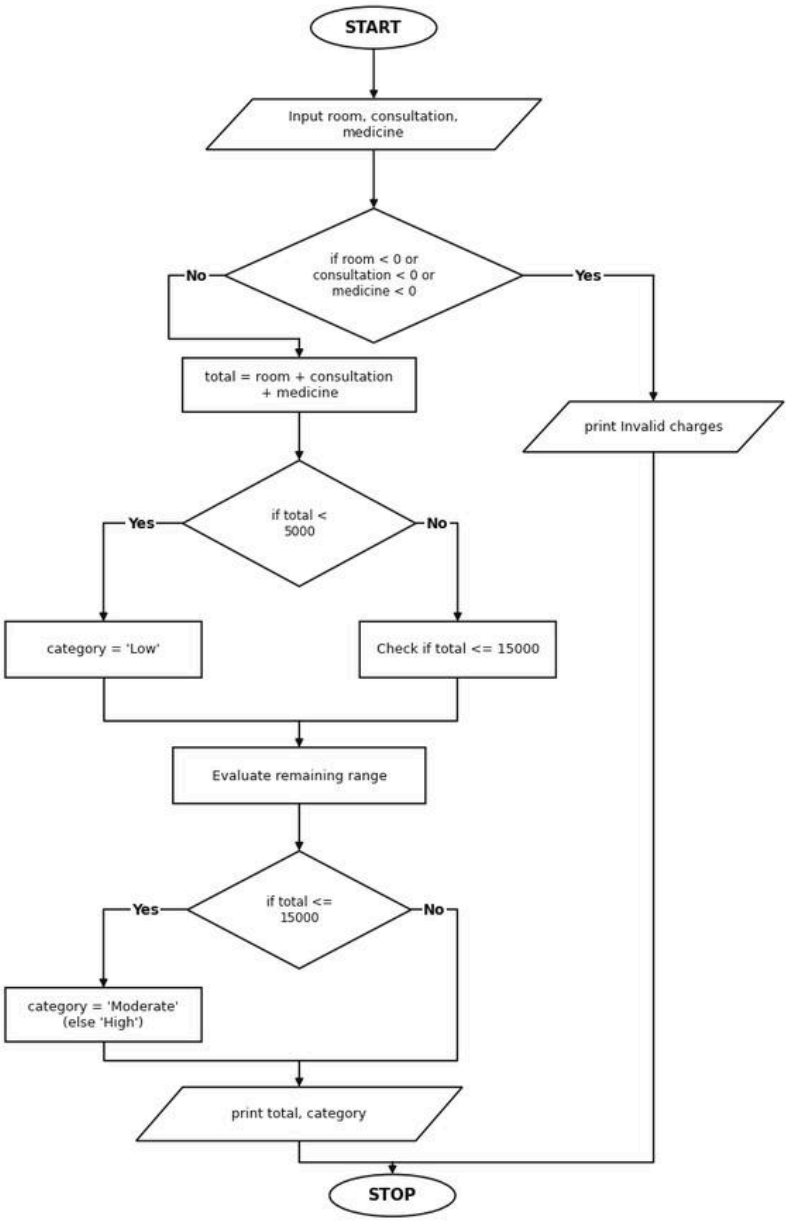
Assumptions

- All entered charges must be zero or positive.
- The classification uses the final total bill amount.

Algorithm

1. Start.
2. Read room, consultation, and medicine charges.
3. Reject any negative charge.
4. Add all charges to compute total.
5. Classify the total into Low, Moderate, or High.
6. Display total and classification.
7. Stop.

Flowchart



Pseudocode

START

INPUT room, consultation, medicine

IF room < 0 OR consultation < 0 OR medicine < 0

  PRINT "Invalid charges"

ELSE

  total = room + consultation + medicine

  IF total < 5000

    category = "Low"

```
    ELSE IF total <= 15000
        category = "Moderate"
    ELSE
        category = "High"
    PRINT total, category
STOP
```

Source Code

```
room = float(input("Enter room charges: "))
consultation = float(input("Enter consultation fee: "))
medicine = float(input("Enter medicine cost: "))

if room < 0 or consultation < 0 or medicine < 0:
    print("Invalid charges")
else:
    total = room + consultation + medicine

    if total < 5000:
        category = "Low"
    elif total <= 15000:
        category = "Moderate"
    else:
        category = "High"

    print("Total hospital bill: Rs.", round(total, 2))
    print("Patient bill category:", category)
```

Test Cases

1. Enter room charges: 1000
   Enter consultation fee: 500
   Enter medicine cost: 700
   Output:
   Total hospital bill: Rs. 2200.0
   Patient bill category: Low
2. Enter room charges: 3000
   Enter consultation fee: 1000
   Enter medicine cost: 2000
   Output:

Total hospital bill: Rs. 6000.0

Patient bill category: Moderate

Output

Case_2_problem_3.py x

```python
# Case 2 - Problem 3: Hospital Billing and Patient Classification

room = float(input("Enter room charges: "))
consultation = float(input("Enter consultation fee: "))
medicine = float(input("Enter medicine cost: "))

if room < 0 or consultation < 0 or medicine < 0:
    print("Invalid charges")
else:
    total = room + consultation + medicine

    if total < 5000:
        category = "Low"
    elif total <= 15000:
        category = "Moderate"
    else:
        category = "High"

    print("Total hospital bill: Rs.", round(total, 2))
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_2_problem_3.py
Enter room charges: 3000
Enter consultation fee: 1000
Enter medicine cost: 2000
Total hospital bill: Rs. 6000.0
Patient bill category: Moderate
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program totals hospital expenses and assigns a category based on the final amount.

3.

Examination Result and Grace-Mark Validation: Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

Sol:

Problem statement

Read marks for five subjects, each out of 100. A student passes if every subject has at least 35 marks. If exactly one subject has 30 to 34 marks and all other subjects are passed, allow up to 5 grace marks to bring that subject to 35.

Objectives

- Apply loops, validation, conditions, and simple result processing.
- Identify subjects below the passing threshold of 35 marks.
- Apply the grace-mark rule when exactly one subject is between 30 and 34.

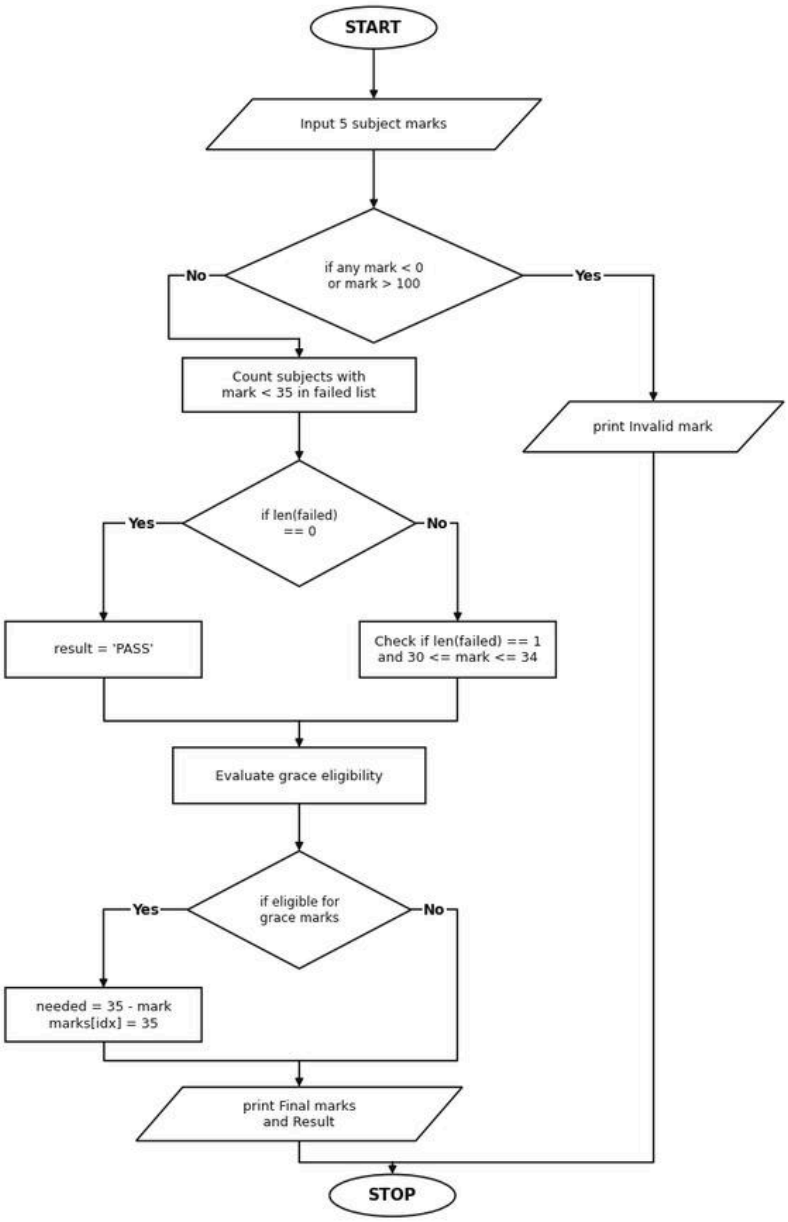
Assumptions

- Marks must be integers from 0 to 100.
- Grace marks are applied to only one subject and are limited to the number needed, up to 5.

Algorithm

1. Start.
2. Read five subject marks and validate each mark.
3. Find subjects with marks below 35.
4. If none are below 35, declare PASS.
5. If exactly one subject has 30 to 34 and the other four passed, add the required grace marks.
6. Otherwise declare FAIL and display final marks and result.
7. Stop.

Flowchart



Pseudocode

START

INPUT 5 subject marks into marks list

IF any mark < 0 OR mark > 100

  PRINT "Invalid mark"

ELSE

  Find indices of marks < 35 in failed list

  IF len(failed) == 0

    result = "PASS"

```
    ELSE IF len(failed) == 1 AND 30 <= marks[failed[0]] <= 34
        needed = 35 - marks[failed[0]]
        marks[failed[0]] = 35
        result = "PASS WITH GRACE MARKS"
    ELSE
        result = "FAIL"
    PRINT marks, result
STOP
```

Source Code

```
marks = []
valid = True

for i in range(5):
    mark = int(input("Enter marks for subject " + str(i + 1) + ": "))
    if mark < 0 or mark > 100:
        print("Invalid mark. Marks must be 0 to 100.")
        valid = False
        break
    marks.append(mark)

if valid:
    failed = []
    for i in range(5):
        if marks[i] < 35:
            failed.append(i)

    if len(failed) == 0:
        result = "PASS"
    elif len(failed) == 1 and 30 <= marks[failed[0]] <= 34:
        index = failed[0]
        needed = 35 - marks[index]
        marks[index] = marks[index] + needed
        result = "PASS WITH " + str(needed) + " GRACE MARKS"
    else:
        result = "FAIL"
```

print("Final marks:", marks)

print("Result:", result)

Test Cases

1. Enter marks for 5 subjects: 70, 65, 32, 80, 50

   Output:

   Final marks: [70, 65, 35, 80, 50]

   Result: PASS WITH 3 GRACE MARKS

2. Enter marks for 5 subjects: 70, 25, 32, 80, 50

   Output:

   Final marks: [70, 25, 32, 80, 50]

   Result: FAIL

Output

Case_3_problem_11.py x

```python
# Case 3 - Problem 11: Examination Result and Grace-Mark Validation

marks = []
valid = True

for i in range(5):
    mark = int(input("Enter marks for subject " + str(i + 1) + ": "))
    if mark < 0 or mark > 100:
        print("Invalid mark. Marks must be 0 to 100.")
        valid = False
        break
    marks.append(mark)

if valid:
    failed = []
    for i in range(5):
        if marks[i] < 35:
            failed.append(i)

```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_3_problem_11.py
Enter marks for subject 1: 70
Enter marks for subject 2: 65
Enter marks for subject 3: 32
Enter marks for subject 4: 80
Enter marks for subject 5: 50
Final marks: [70, 65, 35, 80, 50]
Result: PASS WITH 3 GRACE MARKS
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program validates marks and applies grace marks only when the eligibility rule is satisfied.

4.

Food Waste Monitoring System: Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Sol:

Problem statement

Record food prepared and food wasted for three meals. Calculate total prepared food, total waste, waste percentage, and classify waste as Low (below 10%), Moderate (10% to 25%), or High (above 25%).

Objectives

- Use loops, totals, and conditional classification to monitor food waste.
- Validate that wasted food does not exceed prepared food.
- Calculate overall waste percentage and classify the waste level.

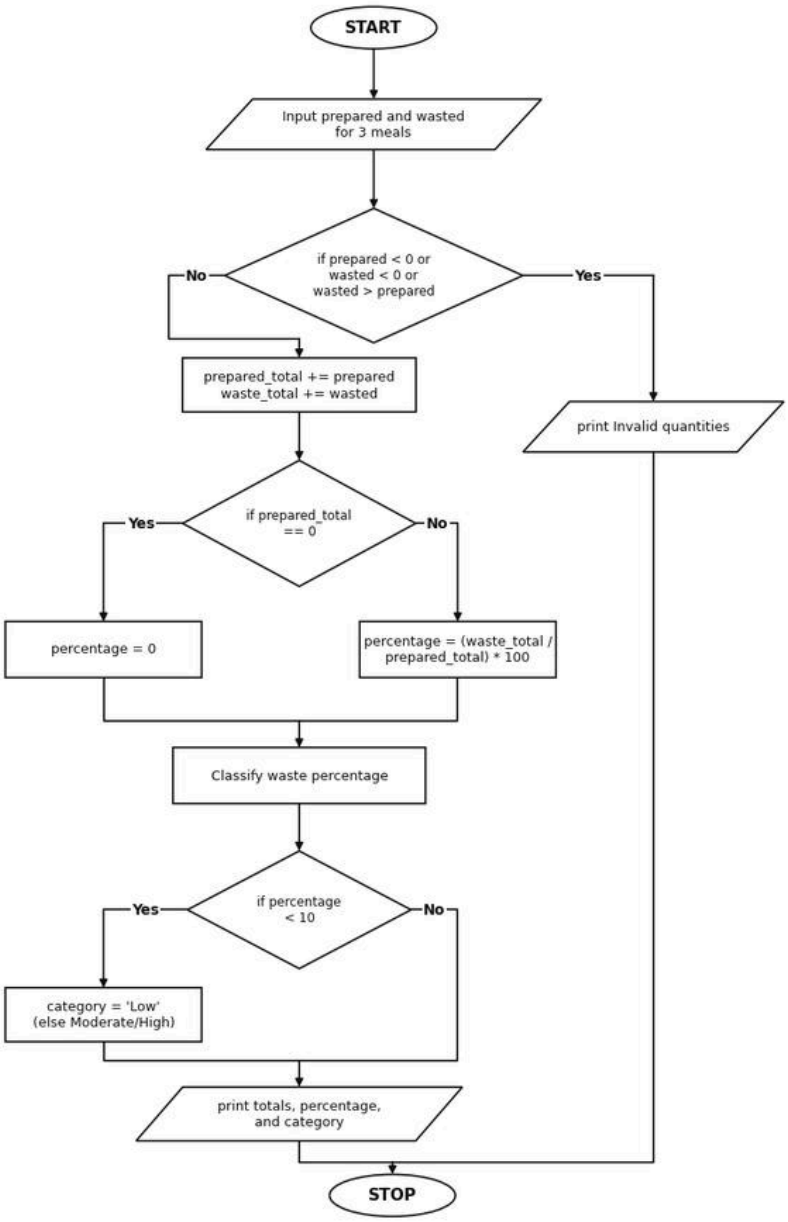
Assumptions

- Quantities are in kilograms and cannot be negative.
- Wasted quantity cannot exceed prepared quantity for any meal.

Algorithm

1. Start.
2. For each of three meals, input prepared and wasted kilograms.
3. Validate the quantities.
4. Add prepared and wasted amounts to totals.
5. Calculate waste percentage as (waste_total / prepared_total) * 100.
6. Classify and display the results.
7. Stop.

Flowchart



Pseudocode

START

SET prepared_total = 0, waste_total = 0

FOR each meal IN ["Breakfast", "Lunch", "Dinner"]

  INPUT prepared, wasted

  IF prepared < 0 OR wasted < 0 OR wasted > prepared

    PRINT "Invalid quantities"

    STOP

  prepared_total = prepared_total + prepared

```
    waste_total = waste_total + wasted
percentage = (waste_total / prepared_total) * 100
CLASSIFY percentage into Low, Moderate, or High
PRINT prepared_total, waste_total, percentage, category
STOP
```

Source Code

```
prepared_total = 0.0
waste_total = 0.0
valid = True

meals = ["Breakfast", "Lunch", "Dinner"]
for meal in meals:
    prepared = float(input(meal + " prepared (kg): "))
    wasted = float(input(meal + " wasted (kg): "))

    if prepared < 0 or wasted < 0 or wasted > prepared:
        print("Invalid quantities")
        valid = False
        break

    prepared_total = prepared_total + prepared
    waste_total = waste_total + wasted

if valid:
    if prepared_total == 0:
        percentage = 0.0
    else:
        percentage = (waste_total / prepared_total) * 100

    if percentage < 10:
        category = "Low"
    elif percentage <= 25:
        category = "Moderate"
    else:
        category = "High"

    print("Total prepared:", round(prepared_total, 2), "kg")
```

print("Total wasted:", round(waste_total, 2), "kg")

print("Waste percentage:", round(percentage, 2), "%")

print("Waste level:", category)

Test Cases

1. Breakfast: 30 kg / 2 kg, Lunch: 45 kg / 6 kg, Dinner: 25 kg / 4 kg

   Output:

   Total prepared: 100.0 kg

   Total wasted: 12.0 kg

   Waste percentage: 12.0 %

   Waste level: Moderate

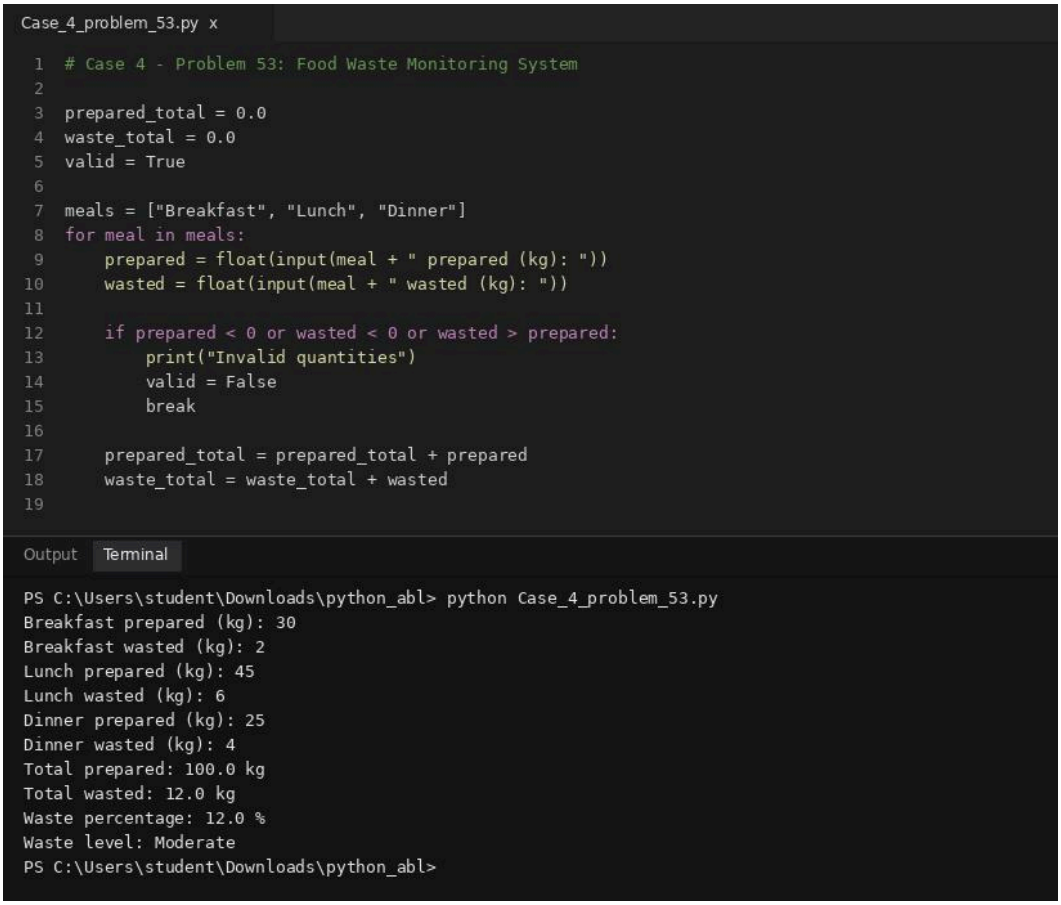
2. Total prepared: 100.0 kg, Total wasted: 5.0 kg

   Output:

   Waste percentage: 5.0 %

   Waste level: Low

Output



Conclusion

The system summarizes food waste and provides a category to support waste reduction.

5.

Traffic Violation Penalty Calculator: Process multiple violations using vehicle category, offence type, repeat-offence status, and predefined fine rules; calculate the final penalty and identify disqualification conditions.

Sol:

Problem statement

Calculate penalties from a list of traffic violations. Sample fines: no helmet Rs. 1,000; signal jump Rs. 2,000; overspeeding Rs. 1,500. Add a 20% surcharge when the total base fine exceeds Rs. 3,000.

Objectives

- Use loops, lists, validation, and conditional calculations.
- Validate each violation name and accumulate the base fine.
- Apply a 20% surcharge when the base fine exceeds Rs. 3,000.

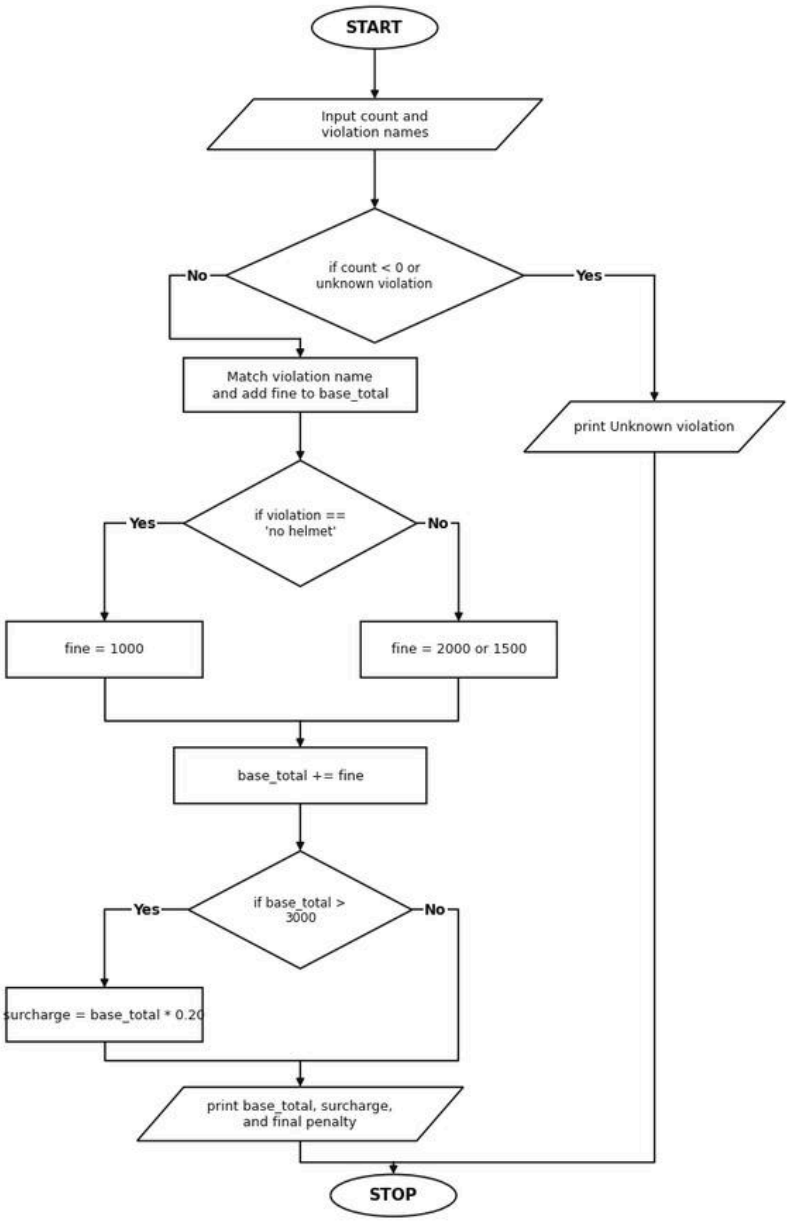
Assumptions

- Only the three listed violation names are accepted.
- Each occurrence is counted separately.
- Surcharge of 20% applies only when base fine > Rs. 3,000.

Algorithm

1. Start.
2. Read the number of violations.
3. For each violation, read the violation name and validate it.
4. Add its fine (1000, 2000, or 1500) to the base total.
5. If base total exceeds Rs. 3,000, calculate a 20% surcharge.
6. Display base total, surcharge, and final total.
7. Stop.

Flowchart



Pseudocode

START

INPUT count

SET base_total = 0

FOR i = 1 TO count

  INPUT violation

  IF violation == "no helmet"

    fine = 1000

  ELSE IF violation == "signal jump"

```
        fine = 2000
    ELSE IF violation == "overspeeding"
        fine = 1500
    ELSE
        PRINT "Unknown violation" and STOP
    base_total = base_total + fine
IF base_total > 3000
    surcharge = base_total * 0.20
ELSE
    surcharge = 0
PRINT base_total, surcharge, base_total + surcharge
STOP
```

Source Code

```
count = int(input("How many violations? "))
base_total = 0
valid = count >= 0

for i in range(count):
    violation = input("Enter violation (no helmet/signal jump/overspeeding): ").strip().lower()
    if violation == "no helmet":
        fine = 1000
    elif violation == "signal jump":
        fine = 2000
    elif violation == "overspeeding":
        fine = 1500
    else:
        print("Unknown violation:", violation)
        valid = False
        break

    print("Fine: Rs.", fine)
    base_total = base_total + fine

if valid:
    if base_total > 3000:
        surcharge = base_total * 0.20
    else:
```

```
    surcharge = 0.0

final_penalty = base_total + surcharge
print("Base fine: Rs.", round(base_total, 2))
print("Surcharge: Rs.", round(surcharge, 2))
print("Final penalty: Rs.", round(final_penalty, 2))
```

Test Cases

1. How many violations? 2 (no helmet, signal jump)

   Output:

   Base fine: Rs. 3000

   Surcharge: Rs. 0.0

   Final penalty: Rs. 3000.0

2. How many violations? 3 (no helmet, signal jump, overspeeding)

   Output:

   Base fine: Rs. 4500

   Surcharge: Rs. 900.0

   Final penalty: Rs. 5400.0

Output

Case_5_problem_21.py x

```python
# Case 5 - Problem 21: Traffic Violation Penalty Calculator

count = int(input("How many violations? "))
base_total = 0
valid = count >= 0

for i in range(count):
    violation = input("Enter violation (no helmet/signal jump/overspeeding): ").strip().lower()
    if violation == "no helmet":
        fine = 1000
    elif violation == "signal jump":
        fine = 2000
    elif violation == "overspeeding":
        fine = 1500
    else:
        print("Unknown violation:", violation)
        valid = False
        break

```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_5_problem_21.py
How many violations? 3
Enter violation (no helmet/signal jump/overspeeding): overspeeding
signal jump
no helmet
Fine: Rs. 1000
Enter violation (no helmet/signal jump/overspeeding): Fine: Rs. 2000
Enter violation (no helmet/signal jump/overspeeding): Fine: Rs. 1500
Base fine: Rs. 4500
Surcharge: Rs. 900.0
Final penalty: Rs. 5400.0
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The calculator totals violations and applies the surcharge only when the base fine exceeds Rs. 3,000.

6.

Student Timetable Conflict Detection: Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

Sol:

Problem statement

Given class records with day, start time, end time, and subject, detect whether any two classes overlap on the same day.

Objectives

- Practice lists, tuples, nested loops, comparisons, and validation.
- Validate that class start and end hours fall within 0 to 24 with start < end.

- Detect and report overlapping classes scheduled on the same day.

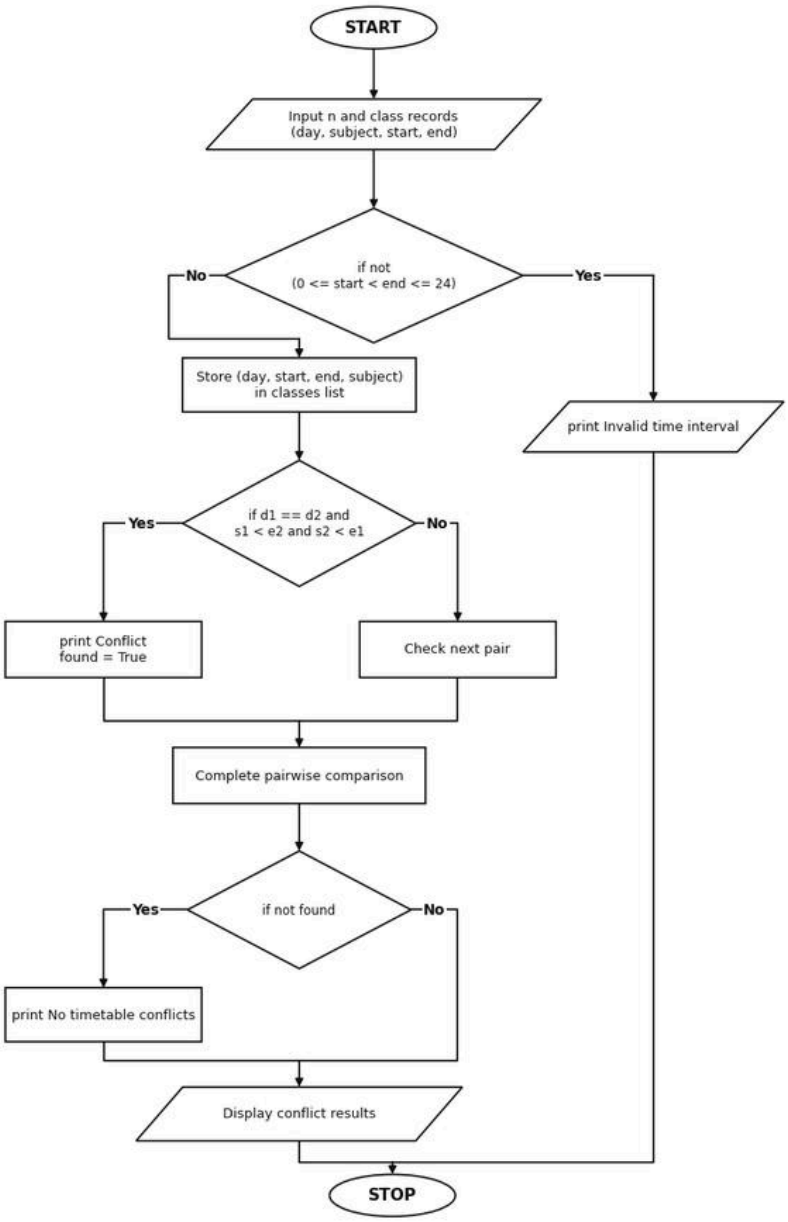
Assumptions

- Times are entered as integer hours in 24-hour format.
- Each class has start < end.
- Two classes overlap if start1 < end2 and start2 < end1 on the same day.

Algorithm

1. Start.
2. Read the number of classes and each class record.
3. Validate start and end times.
4. Compare each pair of classes using nested loops.
5. If two classes occur on the same day and their intervals overlap, report a conflict.
6. If no pair overlaps, report no conflicts and stop.

Flowchart



Pseudocode

START

INPUT n class records (day, start, end, subject)

VALIDATE 0 <= start < end <= 24

conflict = False

FOR i FROM 0 TO n - 1

  FOR j FROM i + 1 TO n - 1

    IF same day AND start_i < end_j AND start_j < end_i

      PRINT conflict

```
            conflict = True
IF NOT conflict
    PRINT "No timetable conflicts"
STOP
```

Source Code

```
classes = []
n = int(input("Enter number of classes: "))
valid = n >= 0

for i in range(n):
    print("Class", i + 1)
    day = input("Day: ").strip().lower()
    subject = input("Subject: ").strip()
    start = int(input("Start hour (0-23): "))
    end = int(input("End hour (1-24): "))

    if not (0 <= start < end <= 24):
        print("Invalid time interval")
        valid = False
        break
    classes.append((day, start, end, subject))

if valid:
    found = False
    for i in range(n):
        for j in range(i + 1, n):
            d1, s1, e1, sub1 = classes[i]
            d2, s2, e2, sub2 = classes[j]
            if d1 == d2 and s1 < e2 and s2 < e1:
                print("Conflict:", sub1, "and", sub2, "on", d1)
                found = True

    if not found:
        print("No timetable conflicts")
```

Test Cases

1. Class 1: mon, Maths, 9 to 11 | Class 2: mon, Python, 10 to 12

   Output:

   Conflict: Maths and Python on mon

2. Class 1: mon, Maths, 9 to 10 | Class 2: mon, Python, 10 to 11

   Output:

   No timetable conflicts

Output

Case_6_problem_23.py x

```python
# Case 6 - Problem 23: Student Timetable Conflict Detection

classes = []
n = int(input("Enter number of classes: "))
valid = n >= 0

for i in range(n):
    print("Class", i + 1)
    day = input("Day: ").strip().lower()
    subject = input("Subject: ").strip()
    start = int(input("Start hour (0-23): "))
    end = int(input("End hour (1-24): "))

    if not (0 <= start < end <= 24):
        print("Invalid time interval")
        valid = False
        break
    classes.append((day, start, end, subject))

```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_6_problem_23.py
Enter number of classes: 3
Class 1
Day: tue
mon
mon
Subject: Physics
Python
Maths
Start hour (0-23): 9
10
9
End hour (1-24): 11
12
11
Class 2
Day: Subject: Start hour (0-23): End hour (1-24): Class 3
Day: Subject: Start hour (0-23): End hour (1-24): Conflict: Maths and Python on mon
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program compares pairs of class intervals and reports overlapping classes on the same day.

7.

Library Borrowing and Fine Reconciliation: Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

Sol:

Problem statement

For each returned book, calculate a late fine of Rs. 2 per day for the first 5 late days and Rs. 5 per day for each additional late day. Sum fines for all books.

Objectives

- Use loops, conditional calculations, and accumulated totals.
- Validate that late days are non-negative integers.
- Calculate book-wise fines and the total library fine.

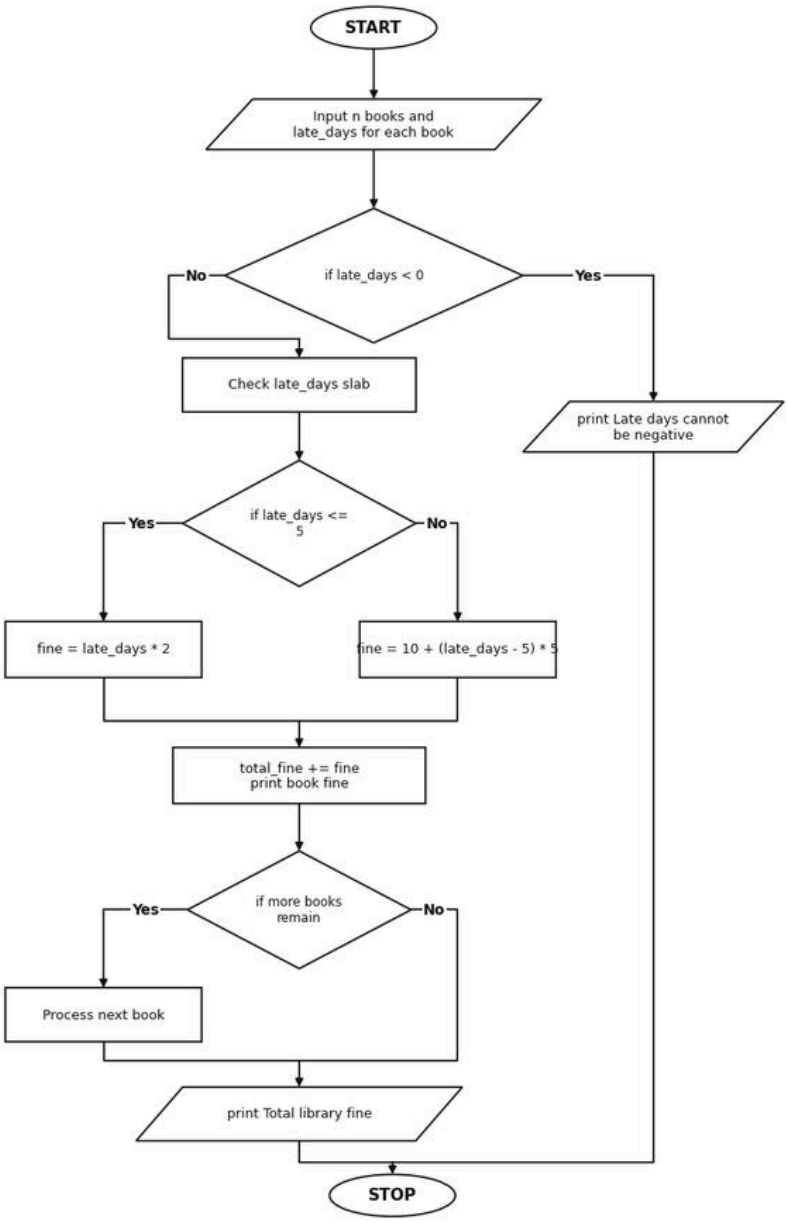
Assumptions

- Late days are non-negative whole numbers.
- No fine is charged when a book is returned on time (0 late days).

Algorithm

1. Start.
2. Read the number of returned books.
3. For each book, read late days and validate.
4. Charge Rs. 2 per day for up to 5 days; for days beyond 5, charge Rs. 5 per extra day.
5. Add each book's fine to the total.
6. Display individual and total fines, then stop.

Flowchart



Pseudocode

START

INPUT number_of_books

SET total_fine = 0

FOR each book

  INPUT late_days

  IF late_days < 0

    PRINT "Late days cannot be negative" and STOP

  ELSE IF late_days <= 5

```
        fine = late_days * 2
    ELSE
        fine = 5 * 2 + (late_days - 5) * 5
    total_fine = total_fine + fine
    PRINT fine
PRINT total_fine
STOP
```

Source Code

```
n = int(input("Number of returned books: "))
total_fine = 0
valid = n >= 0

for i in range(n):
    late_days = int(input("Late days for book " + str(i + 1) + ": "))

    if late_days < 0:
        print("Late days cannot be negative")
        valid = False
        break
    elif late_days <= 5:
        fine = late_days * 2
    else:
        fine = 5 * 2 + (late_days - 5) * 5

    print("Fine for book " + str(i + 1) + ": Rs.", fine)
    total_fine = total_fine + fine

if valid:
    print("Total library fine: Rs.", total_fine)
```

Test Cases

1. Number of returned books: 2 | Book 1: 4 late days | Book 2: 8 late days
   Output:
   Fine for book 1: Rs. 8
   Fine for book 2: Rs. 25
   Total library fine: Rs. 33

2. Number of returned books: 1 | Book 1: 0 late days

   Output:

   Fine for book 1: Rs. 0

   Total library fine: Rs. 0

Output

Case_7_problem_58.py x

```python
# Case 7 - Problem 58: Library Borrowing and Fine Reconciliation

n = int(input("Number of returned books: "))
total_fine = 0
valid = n >= 0

for i in range(n):
    late_days = int(input("Late days for book " + str(i + 1) + ": "))

    if late_days < 0:
        print("Late days cannot be negative")
        valid = False
        break
    elif late_days <= 5:
        fine = late_days * 2
    else:
        fine = 5 * 2 + (late_days - 5) * 5

    print("Fine for book " + str(i + 1) + ": Rs.", fine)
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_7_problem_58.py
Number of returned books: 2
Late days for book 1: 4
Fine for book 1: Rs. 8
Late days for book 2: 8
Fine for book 2: Rs. 25
Total library fine: Rs. 33
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program calculates progressive fines for each book and reconciles them into one total.

8.

Tuple-Based Course Registration Validation: Represent course registrations as tuples, identify duplicate registrations and invalid course combinations, and determine students whose selections violate supplied rules.

Sol:

Problem statement

Store course registrations as tuples containing student ID, course code, and credits. Check for duplicate student-course registrations and calculate total credits for each student.

Objectives

- Use tuples, lists, loops, and duplicate detection.
- Validate student ID, course code, and positive credit values.
- Detect duplicate student-course pairs and sum credits per student.

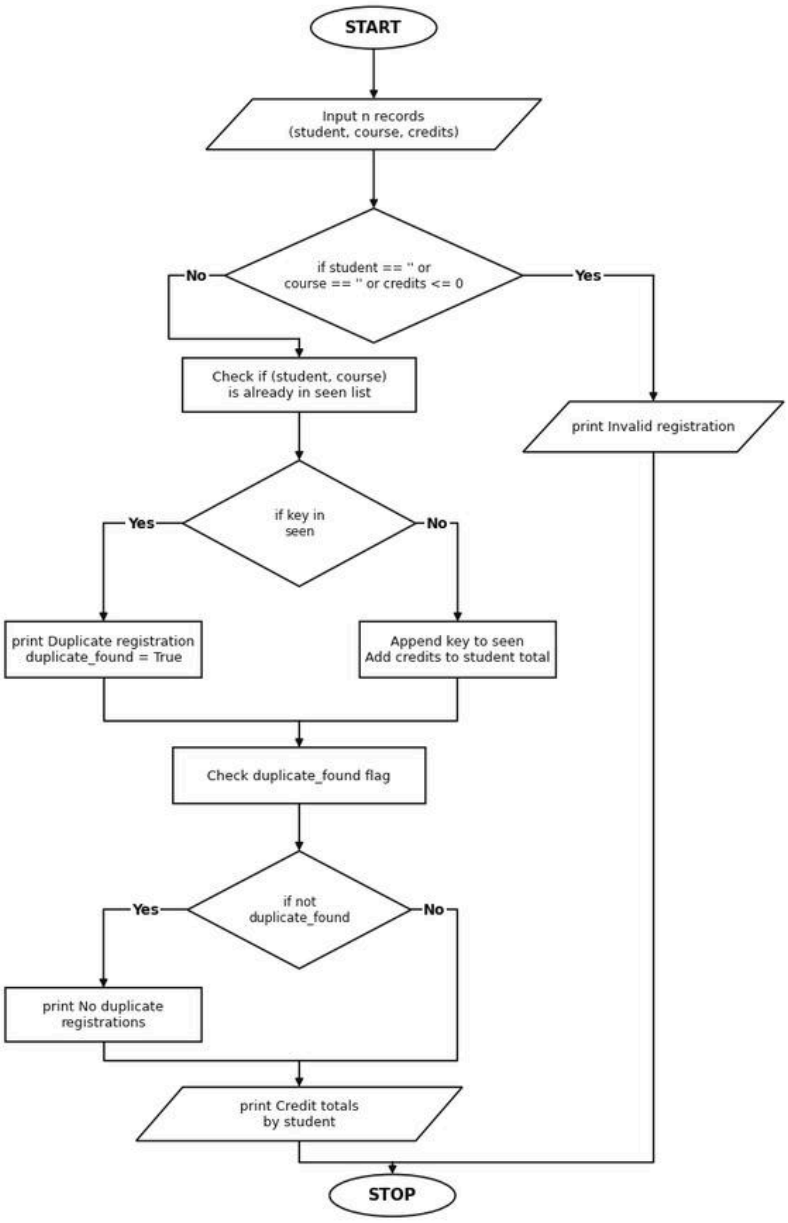
Assumptions

- Each record has a non-empty student ID and course code, and credits must be positive integers.
- A student may register for different courses but not the same course twice.

Algorithm

1. Start.
2. Read the number of registration records.
3. Store each valid record as a tuple (student, course, credits).
4. Compare records to detect repeated (student, course) pairs.
5. Accumulate valid credits per student using lists.
6. Display duplicate status and credit totals per student, then stop.

Flowchart



Pseudocode

START

INPUT n records (student_id, course_code, credits)

VALIDATE fields

seen = empty list

FOR each (student, course, credits) IN registrations

  key = (student, course)

  IF key IN seen

    PRINT "Duplicate registration"

    ELSE
        Append key to seen
        Add credits to student total
PRINT duplicate status and credit totals
STOP

Source Code

```
registrations = []
n = int(input("Number of registration records: "))
valid = n >= 0

for i in range(n):
    student = input("Student ID: ").strip()
    course = input("Course code: ").strip().upper()
    credits = int(input("Credits: "))

    if student == "" or course == "" or credits <= 0:
        print("Invalid registration")
        valid = False
        break
    registrations.append((student, course, credits))

if valid:
    seen = []
    student_ids = []
    student_credits = []
    duplicate_found = False

    for rec in registrations:
        student, course, credits = rec
        key = (student, course)
        if key in seen:
            print("Duplicate registration:", student, "-", course)
            duplicate_found = True
        else:
            seen.append(key)
            if student not in student_ids:
                student_ids.append(student)
```

```
            student_credits.append(credits)
        else:
            idx = student_ids.index(student)
            student_credits[idx] = student_credits[idx] + credits

if not duplicate_found:
    print("No duplicate registrations")

print("Credit totals by student:")
for i in range(len(student_ids)):
    print(student_ids[i], ":", student_credits[i])
```

Test Cases

1. Records: (S01, CS101, 3), (S01, MA101, 4), (S01, CS101, 3)
   Output:
   Duplicate registration: S01 - CS101
   Credit totals by student:
   S01 : 7
2. Records: (S01, CS101, 3), (S01, MA101, 4)
   Output:
   No duplicate registrations
   Credit totals by student:
   S01 : 7

Output

Case_8_problem_64.py x

```python
# Case 8 - Problem 64: Tuple-Based Course Registration Validation

registrations = []
n = int(input("Number of registration records: "))
valid = n >= 0

for i in range(n):
    student = input("Student ID: ").strip()
    course = input("Course code: ").strip().upper()
    credits = int(input("Credits: "))

    if student == "" or course == "" or credits <= 0:
        print("Invalid registration")
        valid = False
        break
    registrations.append((student, course, credits))

if valid:
    seen = []
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_8_problem_64.py
Number of registration records: 3
Student ID: S01
S01
S01
Course code: CS101
MA101
CS101
Credits: 3
4
3
Student ID: Course code: Credits: Student ID: Course code: Credits: Duplicate registration: S01 - CS101
Credit totals by student:
S01 : 7
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program uses tuples to represent registrations and detects duplicate entries while calculating credit totals.

9.

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

Sol:

Problem statement

Allocate a limited number of scholarships to eligible students. Eligibility requires marks of at least 85 and family income no greater than Rs. 300,000. Rank eligible students by marks descending, then lower income ascending, then student ID alphabetically, and award scholarships up to available seats.

Objectives

- Combine record handling, eligibility rules, sorting, tie-breaking, and constrained allocation.
- Filter eligible students meeting marks >= 85 and income <= 300,000.
- Rank candidates and allocate scholarships up to the seat limit.

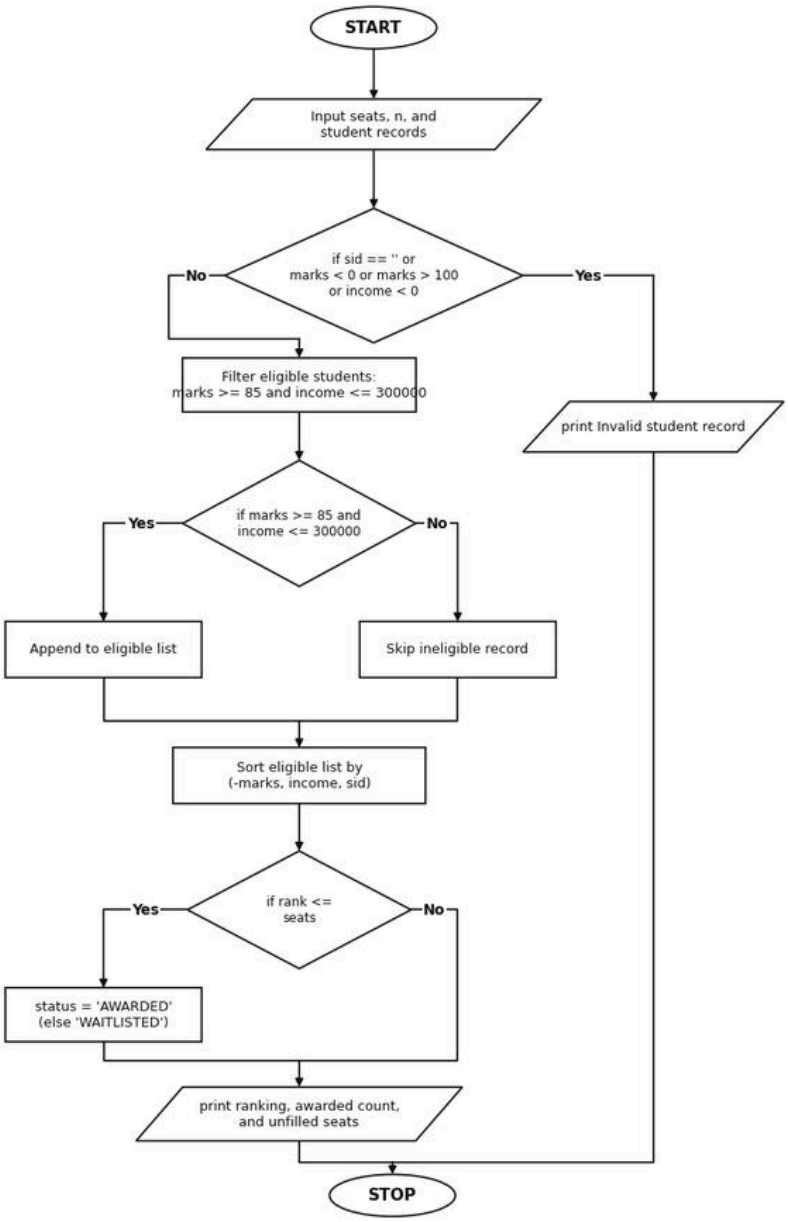
Assumptions

- Each student record contains ID, marks, and annual family income.
- Marks must be 0 to 100 and income must be non-negative.

Algorithm

1. Start.
2. Read scholarship seats and student records.
3. Validate all inputs.
4. Keep students meeting both eligibility conditions (marks >= 85 and income <= 300,000).
5. Sort eligible students by marks descending, income ascending, and ID ascending.
6. Award scholarships to the top candidates up to available seats.
7. Display ranking, award status, and unfilled seats, then stop.

Flowchart



Pseudocode

START

INPUT seats and n student records (sid, marks, income)

VALIDATE records

eligible = students with marks >= 85 AND income <= 300000

SORT eligible by (-marks, income, sid)

FOR rank FROM 1 TO len(eligible)

  IF rank <= seats

    status = "AWARDED"

```
    ELSE
        status = "WAITLISTED / NOT AWARDED"
    PRINT rank, sid, marks, income, status
PRINT awarded count and unfilled seats
STOP
```

Source Code

```
seats = int(input("Number of scholarships available: "))
n = int(input("Number of students: "))
students = []
valid = seats >= 0 and n >= 0

for i in range(n):
    sid = input("Student ID: ").strip()
    marks = float(input("Marks (0-100): "))
    income = float(input("Annual family income: "))

    if sid == "" or not (0 <= marks <= 100) or income < 0:
        print("Invalid student record")
        valid = False
        break
    students.append((sid, marks, income))

if valid:
    eligible = []
    for s in students:
        sid, marks, income = s
        if marks >= 85 and income <= 300000:
            eligible.append((-marks, income, sid, marks))

    eligible.sort()

    print("Scholarship ranking:")
    rank = 1
    awarded_count = 0
    for item in eligible:
        neg_m, income, sid, marks = item
        if rank <= seats:
```

```
        status = "AWARDED"
        awarded_count = awarded_count + 1
    else:
        status = "WAITLISTED / NOT AWARDED"
    print(rank, sid, marks, "Rs.", round(income, 2), status)
    rank = rank + 1

if seats > awarded_count:
    unfilled = seats - awarded_count
else:
    unfilled = 0

print("Scholarships awarded:", awarded_count)
print("Unfilled seats:", unfilled)
```

Test Cases

1. Seats: 2 | Students: A (90, 250000), B (95, 290000), C (88, 200000)
   Output:
   1 B 95.0 Rs. 290000.0 AWARDED
   2 A 90.0 Rs. 250000.0 AWARDED
   3 C 88.0 Rs. 200000.0 WAITLISTED / NOT AWARDED
   Scholarships awarded: 2
   Unfilled seats: 0

Output

Case_9_problem_1.py x

```python
# Case 9 - Problem 1: University Scholarship Allocation

seats = int(input("Number of scholarships available: "))
n = int(input("Number of students: "))
students = []
valid = seats >= 0 and n >= 0

for i in range(n):
    sid = input("Student ID: ").strip()
    marks = float(input("Marks (0-100): "))
    income = float(input("Annual family income: "))

    if sid == "" or not (0 <= marks <= 100) or income < 0:
        print("Invalid student record")
        valid = False
        break
    students.append((sid, marks, income))

if valid:
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_9_problem_1.py
Number of scholarships available: 2
Number of students: 3
Student ID: C
B
A
Marks (0-100): 88
95
90
Annual family income: 200000
290000
250000
Student ID: Marks (0-100): Annual family income: Student ID: Marks (0-100): Annual family income: Schol
1 B 95.0 Rs. 290000.0 AWARDED
2 A 90.0 Rs. 250000.0 AWARDED
3 C 88.0 Rs. 200000.0 WAITLISTED / NOT AWARDED
Scholarships awarded: 2
Unfilled seats: 0
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The program applies eligibility rules and deterministic tie-breaking to allocate a limited number of scholarships.

10.

University Admission Decision Engine: Evaluate candidates using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria; generate a ranked list of eligible applicants.

Sol:

Problem statement

Determine admission eligibility using entrance score, qualifying-exam percentage, and available seats. A candidate is eligible if entrance score is at least 70 and qualifying percentage is at least 60. Rank

eligible candidates by entrance score descending, qualifying percentage descending, then application ID ascending, and admit candidates up to the seat limit.

Objectives

- Integrate input validation, filtering, multi-key ranking, and limited-seat allocation.
- Filter applicants meeting entrance >= 70 and qualifying >= 60.
- Rank eligible candidates and assign ADMITTED or WAITLISTED status.

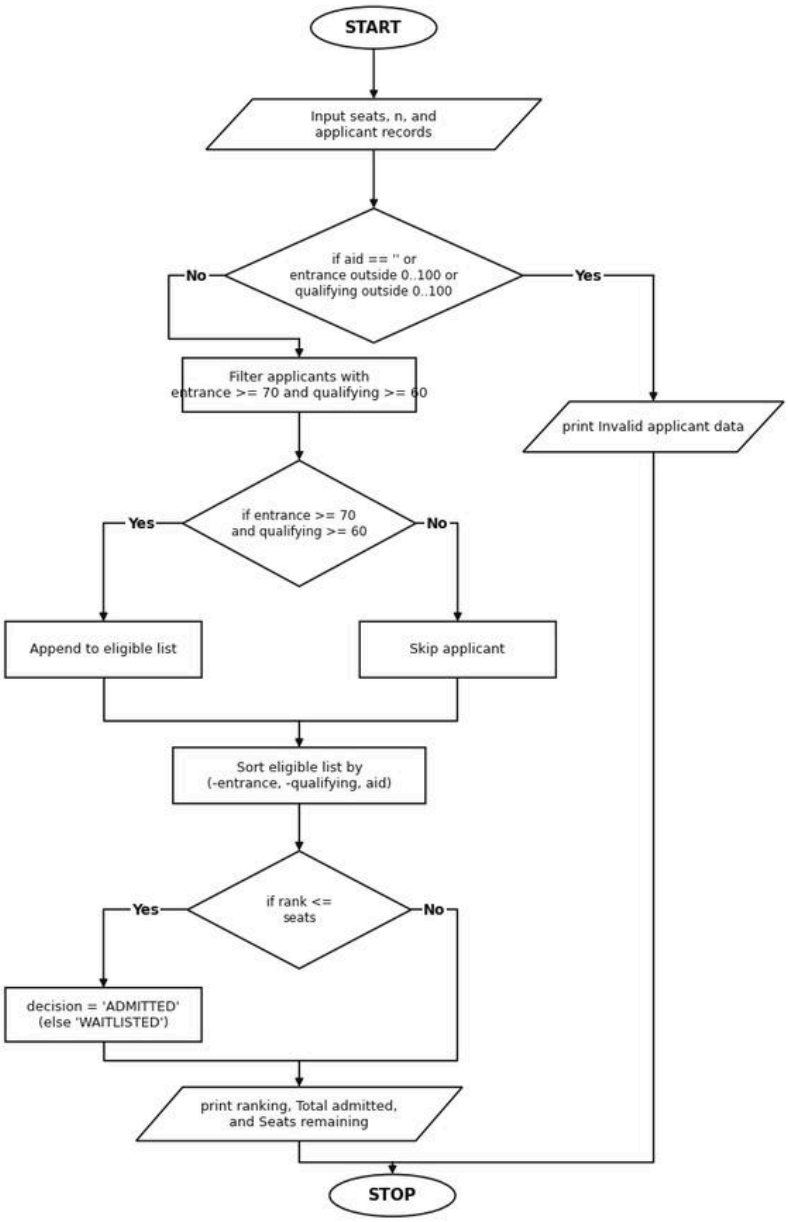
Assumptions

- Scores are from 0 to 100 and seat count is non-negative.
- Ties in entrance score are broken by qualifying percentage descending, then ID ascending.

Algorithm

1. Start.
2. Read seat capacity and candidate records.
3. Validate IDs, scores, and percentages.
4. Filter candidates meeting both minimum criteria.
5. Sort by entrance score descending, qualifying percentage descending, and ID ascending.
6. Admit candidates up to seat capacity.
7. Display each eligible candidate's rank and decision, then stop.

Flowchart



Pseudocode

START

INPUT seats and n candidate records (aid, entrance, qualifying)

VALIDATE records

eligible = candidates where entrance >= 70 AND qualifying >= 60

SORT eligible by (-entrance, -qualifying, aid)

FOR rank FROM 1 TO len(eligible)

  IF rank <= seats

    decision = "ADMITTED"

```
    ELSE
        decision = "WAITLISTED"
    PRINT rank, aid, entrance, qualifying, decision
PRINT total admitted and seats remaining
STOP
```

Source Code

```
seats = int(input("Available seats: "))
n = int(input("Number of applicants: "))
applicants = []
valid = seats >= 0 and n >= 0

for i in range(n):
    aid = input("Application ID: ").strip()
    entrance = float(input("Entrance score (0-100): "))
    qualifying = float(input("Qualifying percentage (0-100): "))

    if aid == "" or not (0 <= entrance <= 100) or not (0 <= qualifying <= 100):
        print("Invalid applicant data")
        valid = False
        break
    applicants.append((aid, entrance, qualifying))

if valid:
    eligible = []
    for a in applicants:
        aid, entrance, qualifying = a
        if entrance >= 70 and qualifying >= 60:
            eligible.append((-entrance, -qualifying, aid, entrance, qualifying))

    eligible.sort()

    print("Admission results:")
    rank = 1
    admitted_count = 0
    for item in eligible:
        neg_e, neg_q, aid, entrance, qualifying = item
        if rank <= seats:
```

```
        decision = "ADMITTED"
        admitted_count = admitted_count + 1
    else:
        decision = "WAITLISTED"
    print(rank, aid, entrance, qualifying, decision)
    rank = rank + 1

if seats > admitted_count:
    rem_seats = seats - admitted_count
else:
    rem_seats = 0

print("Total admitted:", admitted_count)
print("Seats remaining:", rem_seats)
```

Test Cases

1. Seats: 2 | Applicants: A (80, 80), B (90, 70), C (85, 90)
   Output:
   1 B 90.0 70.0 ADMITTED
   2 C 85.0 90.0 ADMITTED
   3 A 80.0 80.0 WAITLISTED
   Total admitted: 2
   Seats remaining: 0

Output

Case_10_problem_20.py x

```python
# Case 10 - Problem 20: University Admission Decision Engine

seats = int(input("Available seats: "))
n = int(input("Number of applicants: "))
applicants = []
valid = seats >= 0 and n >= 0

for i in range(n):
    aid = input("Application ID: ").strip()
    entrance = float(input("Entrance score (0-100): "))
    qualifying = float(input("Qualifying percentage (0-100): "))

    if aid == "" or not (0 <= entrance <= 100) or not (0 <= qualifying <= 100):
        print("Invalid applicant data")
        valid = False
        break
    applicants.append((aid, entrance, qualifying))

if valid:
```

Output Terminal

```
PS C:\Users\student\Downloads\python_abl> python Case_10_problem_20.py
Available seats: 2
Number of applicants: 3
Application ID: C
B
A
Entrance score (0-100): 85
90
80
Qualifying percentage (0-100): 90
70
80
Application ID: Entrance score (0-100): Qualifying percentage (0-100): Application ID: Entrance score (
1 B 90.0 70.0 ADMITTED
2 C 85.0 90.0 ADMITTED
3 A 80.0 80.0 WAITLISTED
Total admitted: 2
Seats remaining: 0
PS C:\Users\student\Downloads\python_abl>
```

Conclusion

The system filters candidates, ranks them consistently, and respects the available seat limit.